Xử Lý Ảnh và Thị Giác Máy Tính

Bài 1 — Từ Điểm Ảnh Đến Phép Lọc

Môn học 121036

Trường Đại Học Giao Thông Vận Tải TP. Hồ Chí Minh

Về tôi

- Võ Thượng Anh
- 2014-2018: Cử nhân Đại học Creighton, chuyên ngành toán học và vật lý
- 2018-2024: Tiến sĩ, Đại học Lincoln-Nebraska, nghiên cứu về PDE/mô hình phi địa phương.



Thông tin về lớp

1 buổi lý thuyết và 1 buổi vấn đáp thực hành

Lộ trình:

- Buổi 1+2: Chương 2: Xử lý ảnh
- Buổi 3+4: Chương 3: Trích xuất đặc trưng
- Buổi 5+6: Chương 4: Phân vùng
- Buổi 7+8: Chương 5: Nhận dạng ảnh
- Buổi 9+10: Bài tập lớn/Dự án

Đánh giá

- Tham gia (10%)
- Bài tập cuối chương 40%
- Bài tập lớn (50%)

Về phần vấn đáp:

- Chỉ được sử dụng 1 thiết bị điện tử 1 lúc (1 máy tính), ngoài ra được phép sử dụng 1 tờ giấy A4
- Vấn đáp chiếm 50% số điểm bài tập

Yêu cầu vấn đáp bài tập cuối chương:

- Mục tiêu và Mục đích (Objective & Purpose)
- Logic Thuật toán (Algorithmic Logic)
- Tư duy Máy tính (Computational Thinking)
- Nhận xét về kết quả, liên kết với bài học

Bạn thấy thị giác máy tính ở đâu?

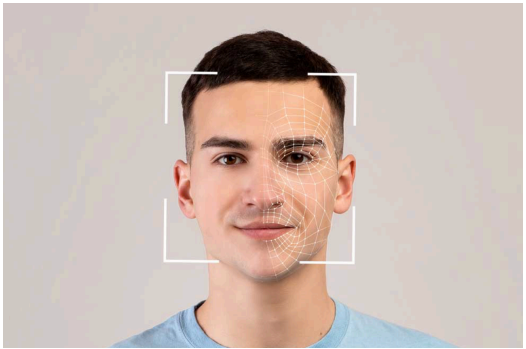


Figure: Mở khóa điện thoại



Figure: Phát hiện làn đường, biển báo

Ví dụ khác trong cuộc sống

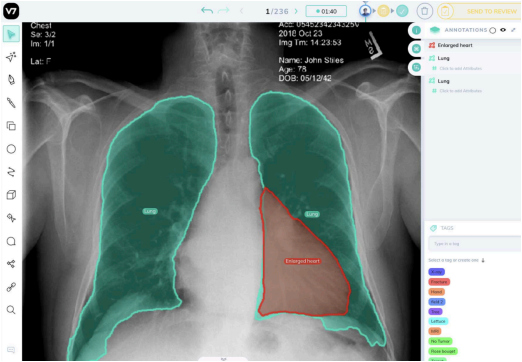


Figure: Chẩn đoán y khoa



Figure: Quét mã QR

Thị giác máy tính là gì?

Định nghĩa

Thị giác máy tính cho phép máy tính "**nhìn**" và "**hiểu**" hình ảnh như con người

Cách thị giác máy tính hoạt động

1. **Thu nhận**: Camera/Sensor
2. **Xử lý**: Filter, Transform
3. **Hiểu**: Detect, Recognize
4. **Quyết định**: Action

Ví dụ: Xe tự lái

1. Thu: Camera quay đường
2. Xử lý: Tăng độ tương phản
3. Hiểu: Phát hiện làn đường
4. Quyết định: Giữ xe trong làn

Ví dụ: Face ID

1. Thu: Camera chụp khuôn mặt
2. Xử lý: Chuẩn hóa ánh sáng
3. Hiểu: So sánh với dữ liệu
4. Quyết định: Mở khóa / Từ chối

Xử lý ảnh vs Thị giác máy tính

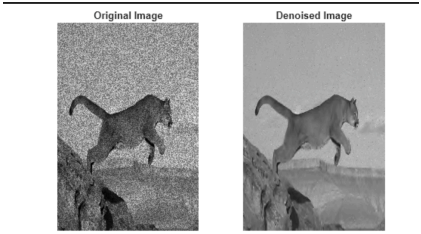


Figure: Xử lý ảnh

Mục tiêu: Ảnh → Ảnh tốt hơn

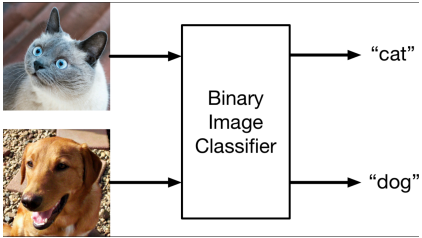
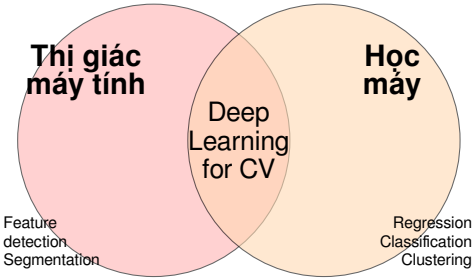


Figure: Thị giác máy tính

Mục tiêu: Ảnh → Thông tin/Quyết định

Xử lý ảnh là công cụ để làm thị giác máy tính

Thị giác máy tính vs Học máy



Mối quan hệ:

- Thị giác máy tính *sử dụng* học máy
- Học máy giúp *học từ dữ liệu* thay vì lập trình thủ công
- Deep learning = phương pháp học máy hiện đại cho CV

Ví dụ:

Nhận diện mèo/chó = bài toán CV giải bằng học máy

Ứng dụng: An ninh và Giám sát

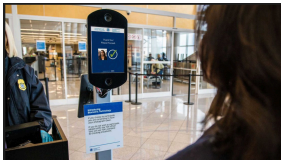


Figure: Nhận dạng khuôn mặt

- Kiểm soát ra vào
- Tìm kiếm người mất tích
- Xác thực danh tính

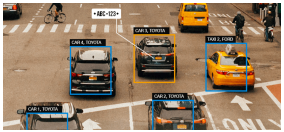


Figure: Nhận dạng biển số

- Thu phí tự động
- Giám sát giao thông
- Phát hiện vi phạm

Ứng dụng: Giao thông thông minh



Figure: Xe tự hành

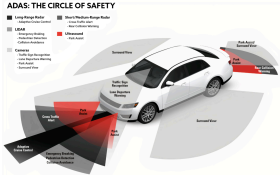


Figure: Hỗ trợ lái xe (ADAS)

- Phát hiện làn đường
- Nhận diện biển báo
- Tránh vật cản

- Cảnh báo va chạm
- Giữ làn đường
- Nhận diện người đi bộ

Ứng dụng: Y khoa

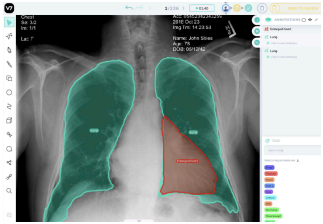


Figure: Phân tích ảnh y khoa

Các tác vụ:

- Phát hiện khối u
- Phân đoạn cơ quan
- Đo kích thước tổn thương
- Theo dõi tiến triển bệnh

Lợi ích:

- Hỗ trợ bác sĩ chẩn đoán
- Phát hiện sớm bệnh lý
- Giảm sai sót

Ứng dụng: Sản xuất và Công nghiệp

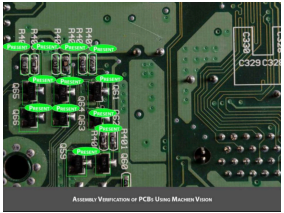


Figure: Kiểm tra chất lượng

- Phát hiện lỗi trên dây chuyền
- Đo kích thước sản phẩm
- Kiểm tra bề mặt

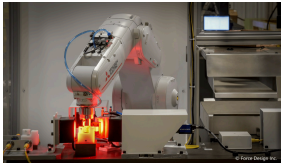


Figure: Robot thị giác

- Nhận diện vị trí linh kiện
- Lắp ráp tự động
- Phân loại sản phẩm

Thách thức 1: Biến đổi hình học



Figure: Cùng một đối tượng, nhiều góc nhìn khác nhau

Câu hỏi: Làm sao máy tính nhận ra đây là cùng một vật?

- Con người: Dễ dàng nhận ra
- Máy tính: Các giá trị pixel hoàn toàn khác nhau!

Thách thức 2: Biến đổi ánh sáng



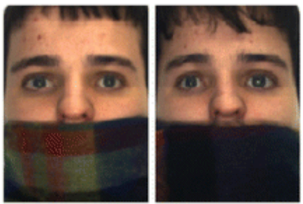
Figure: Cùng cảnh, điều kiện ánh sáng khác nhau

Vấn đề:

- Màu sắc thay đổi
- Bóng đổ che khuất
- Độ tương phản khác nhau

Thách thức 3: Che khuất





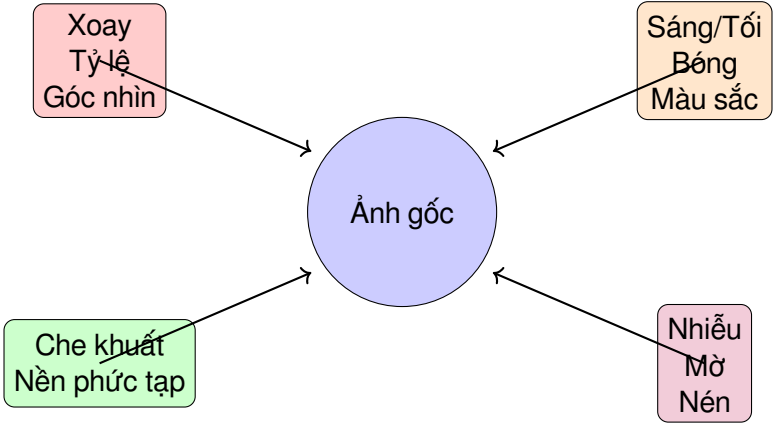
Vấn đề:

- Chỉ nhìn thấy một phần
- Vật khác che khuất
- Cần suy luận phần bị che

Ví dụ thực tế:

- Người đi bộ sau xe
- Khuôn mặt đeo khẩu trang
- Vật thể chồng lên nhau

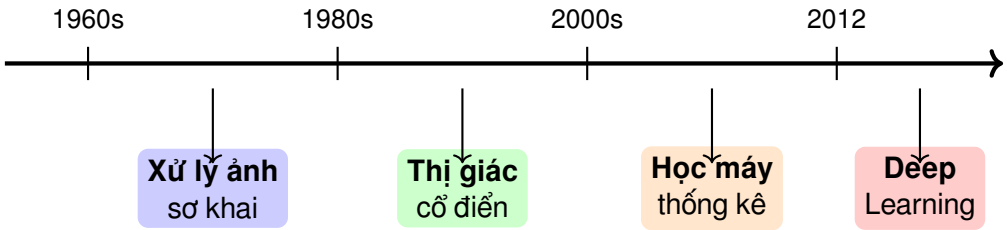
Tóm tắt: Tại sao thị giác máy tính khó?



Mục tiêu của môn học:

Học cách *thiết kế giải pháp* cho từng thách thức

Quá trình phát triển của Thị giác máy tính



4 giai đoạn chính:

1. Xử lý ảnh sơ khai (1960s–1980s)
2. Thị giác máy tính cổ điển (1980s–2000s)
3. Học máy thống kê (2000s–2012)
4. Kỷ nguyên Deep Learning (2012–nay)

Xu hướng:

- Từ thủ công → Tự động học
- Từ đơn giản → Phức tạp
- Từ lý thuyết → Dữ liệu lớn

Giai đoạn 1: Xử lý ảnh sơ khai (1960s–1980s)

Đặc điểm chính:

- Xử lý ảnh cơ bản
- Các phép toán trên pixel
- Lọc tuyến tính
- Phát hiện cạnh đơn giản

Công cụ:

- Toán tử điểm (point operators)
- Convolution
- Histogram equalization
- Thresholding

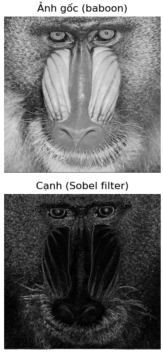


Figure: Ví dụ: Phát hiện cạnh

Ví dụ: Giảm nhiễu trong ảnh (1970s)



Figure: Ví dụ filter

Hạn chế:

- Chỉ xử lý ở mức pixel, không hiểu nội dung
- Cần điều chỉnh tham số thủ công
- Khó tổng quát hóa

Giai đoạn 2: Thị giác máy tính cổ điển (1980s–2000s)

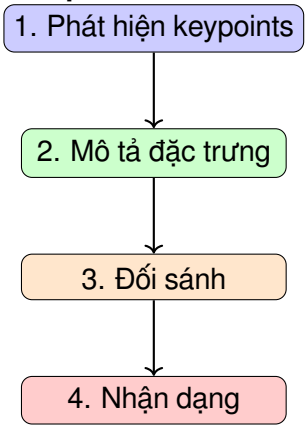
Bước tiến lớn:

- Phát hiện **đặc trưng**
- Mô tả đối tượng
- Đối sánh và nhận dạng
- Hình học 3D

Ý tưởng chính:

Tìm các điểm đặc trưng (features) bất biến với biến đổi hình học và ánh sáng

Pipeline cổ điển:



Các phương pháp quan trọng khác

Phương pháp	Năm	Đóng góp chính	Ứng dụng
Harris Corner	1988	Phát hiện góc bất biến xoay	Tracking, matching
SIFT	1999	Bất biến tỷ lệ và xoay	Panorama, 3D reconstruction
Canny Edge	1986	Phát hiện cạnh tối ưu	Segmentation, contour
Hough Transform	1972	Phát hiện hình dạng	Phát hiện đường, vòng tròn
HOG	2005	Gradient features	Phát hiện người đi bộ

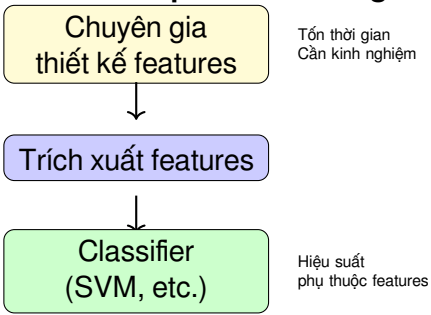
Đặc điểm chung: Thiết kế thủ công (hand-crafted features)

Hạn chế của Thị giác máy tính cổ điển

Vấn đề chính:

1. **Thiết kế thủ công**
   - Cần chuyên gia thiết kế features
   - Mất nhiều thời gian
2. **Khó tổng quát**
   - Features tốt cho bài toán A
   - Chưa chắc tốt cho bài toán B
3. **Hiệu suất hạn chế**
   - Độ chính xác 70-80%
   - Khó đạt mức con người

Pipeline thủ công:



Giai đoạn 3: Học máy thống kê (2000s–2012)

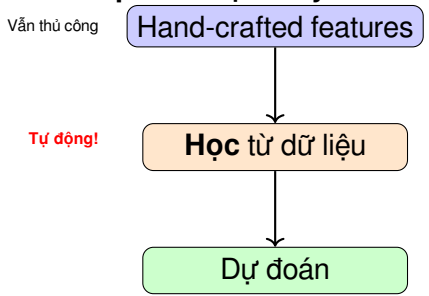
Bước chuyển:

- Vẫn dùng hand-crafted features
- Nhưng **học** cách kết hợp chúng
- Sử dụng dữ liệu nhiều hơn

Phương pháp phổ biến:

- Support Vector Machines (SVM)
- Random Forests
- Boosting (AdaBoost)
- Bag of Visual Words

Pipeline học máy:

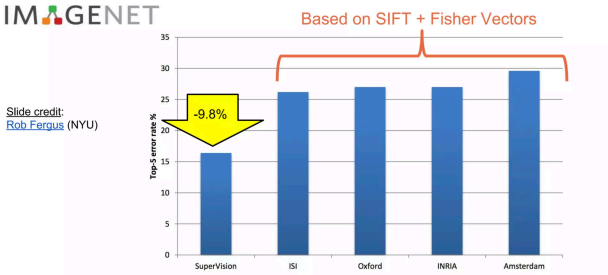


Cải thiện:

- Độ chính xác tăng (85%)
- Tổng quát tốt hơn

ImageNet Large Scale Visual Recognition Challenge (ILSVRC)

ImageNet Challenge: 2012



Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., ... & Fei-Fei, L. (2014). Imagenet large scale visual recognition challenge. *arXiv preprint arXiv:1409.0575*. [web]

Figure: Kết quả thi ImageNet 2012

AlexNet (2012):

- Convolutional Neural Network
- 8 layers (5 conv + 3 FC)
- Giảm error từ 26% $\rightarrow$ 16%
- Sử dụng GPU

Tác động:
Khởi đầu kỷ nguyên Deep Learning cho CV!

Giai đoạn 4: Kỷ nguyên Deep Learning (2012–nay)

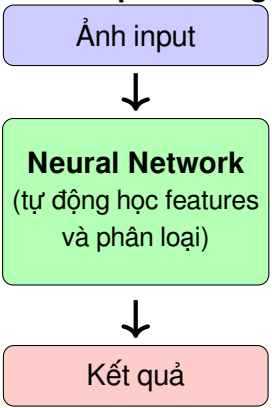
Đặc điểm chính:

- **End-to-end learning**
- Tự động học features
- Không cần thiết kế thủ công
- Dữ liệu lớn + GPU mạnh

Kiến trúc phổ biến:

- CNN (Convolutional Neural Networks)
- ResNet, VGG, Inception
- Transformer (Vision Transformer)

Deep Learning Pipeline:



End-to-end!

Tiến bộ ImageNet 2012–2017

Năm	Mô hình	Top-5 Error
2012	AlexNet	16.4%
2013	ZFNet	11.7%
2014	VGG	7.3%
2014	GoogLeNet	6.7%
2015	ResNet	3.6%
2017	SENet	2.25%
-	Con người	5%

Xu hướng:

- Mô hình ngày càng sâu
- Accuracy tăng nhanh
- Vượt con người (2015)

Yếu tố thành công:

- Dữ liệu lớn (ImageNet)
- GPU mạnh
- Kiến trúc tốt
- Kỹ thuật huấn luyện

Deep Learning trong CV hiện nay

Phân loại ảnh:

- ResNet, EfficientNet
- Accuracy >95% trên ImageNet

Phát hiện đối tượng:

- YOLO, Faster R-CNN
- Real-time detection

Phân đoạn:

- U-Net, Mask R-CNN
- Semantic/Instance segmentation

Ứng dụng mới:

- Tạo ảnh (GANs, Diffusion)
- Style transfer
- Super-resolution
- Video understanding
- 3D từ 2D

Hiện tại

Deep Learning là phương pháp chủ đạo trong Computer Vision

Tại sao vẫn cần học các kiến thức nền tảng?

1. Deep Learning mã hóa các nguyên lý cổ điển:

- Convolution = Filtering
- Pooling = Downsampling
- Skip connections = Multi-scale

2. Hiểu cách hoạt động:

- Tại sao mô hình thất bại?
- Khi nào nên dùng phương pháp nào?
- Làm sao cải thiện?

3. Xây dựng giải pháp mới:

- Kết hợp classical + deep learning
- Thiết kế kiến trúc mới
- Pre-processing thông minh

4. Không phải lúc nào cũng cần DL:

- Dữ liệu ít
- Tài nguyên hạn chế
- Bài toán đơn giản

Nền tảng vững = Hiểu sâu = Sáng tạo tốt hơn!

Liên kết với nội dung môn học

Môn học này tập trung vào nền tảng:

Chương 2: Xử lý ảnh

- Point operators
- Filtering (linear, non-linear)
- Transforms (Fourier, Wavelet)
- Geometric transforms

Chương 3: Features

- Corner, edge detection
- Feature matching
- Classical descriptors

Chương 4: Segmentation

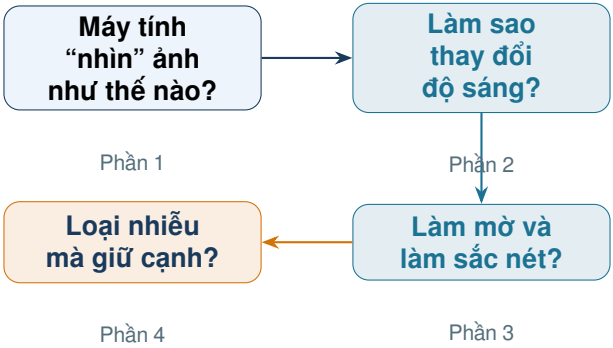
- Active contours
- Split and merge
- Mean-shift, clustering

Chương 5: Recognition

- Object recognition
- Kết nối với deep learning

Sau khi hiểu nền tảng, bạn sẽ dễ dàng học và áp dụng deep learning!

Một Câu Hỏi Xuyên Suốt Bài Học



Triết lý

Mỗi khái niệm bắt đầu từ toán học bạn **đã biết ở Giải Tích 2**, rồi đặt câu hỏi: "Khi biến rời rạc thay thế liên tục, công thức thay đổi ra sao?"

Máy Tính “Nhìn” Ảnh Như Thế Nào?

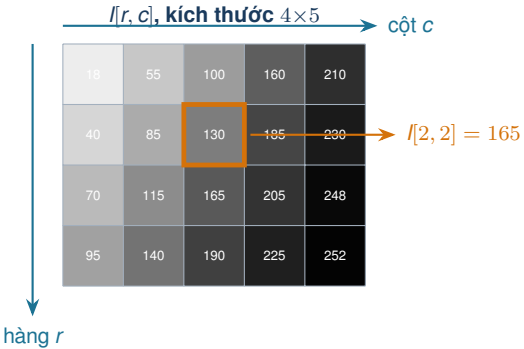
“Một bức ảnh nghìn lời, nhưng với máy tính nó chỉ là một bảng số nguyên.”

Ảnh Là Một Hàm Hai Biến

Liên hệ Giải Tích

Trong Giải Tích, bạn đã học hàm hai biến $f(x, y)$ xác định trên miền liên tục $\mathbb{R}^2$.

Ảnh số là **cùng ý tưởng đó**, nhưng trên miền **rời rạc hữu hạn**.



Hàm ảnh xám:

$$I : \{0, \ldots, H-1\} \times \{0, \ldots, W-1\} \to \{0, \ldots, 255\}$$

- Mỗi cặp (r, c) là một **điểm** trong miền rời rạc
- $I[r, c]$ là **giá trị** tại điểm đó
- "Hàm" này không liên tục — không có giới hạn, không có đạo hàm (chưa!)

Từ Hàm Liên Tục Đến Ảnh Rời Rạc

Quy ước `uint8`:

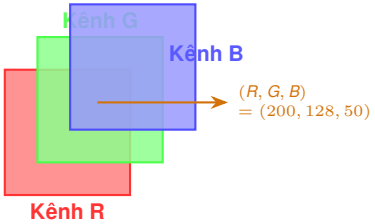
- Giá trị nguyên $I[r, c] \in \{0, 1, \ldots, 255\}$
- $0 =$ đen hoàn toàn; $255 =$ trắng hoàn toàn
- $2^8 = 256$ mức — đủ để mắt người không phân biệt được bước nhảy

Ảnh màu = 3 hàm chồng nhau (kênh RGB):

$$\text{Shape} = H \times W \times 3$$

Lưu ý OpenCV

OpenCV lưu theo thứ tự **BGR**, không phải RGB. Đây là nguồn lỗi rất phổ biến!



Chuyển RGB sang ảnh xám (trọng số luminance):

$$Y = 0.299R + 0.587G + 0.114B$$

Hệ số $\neq \frac{1}{3}$ vì mắt người nhạy với xanh lá nhất.

Demo 1

Ảnh Là Một Hàm Hai Biến Rời Rạc

Tạo mảng màu tổng hợp · In `shape`, `dtype`

Tách 3 kênh · Vẽ mặt cắt $I[r_0, \cdot]$ theo một hàng pixel

Làm Sao Thay Đổi Độ Sáng?

Toán Tử Điểm Ảnh

Hàm Truyền — Cùng Khái Niệm Từ Giải Tích

Liên hệ Giải Tích

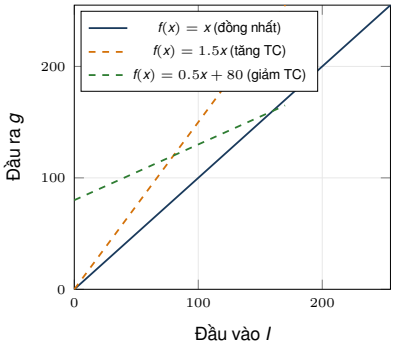
Trong Giải Tích, bạn đã vẽ đồ thị $y = f(x)$ và phân tích hành vi của hàm số.

Toán tử điểm ảnh chính là hàm $f : [0, 255] \to [0, 255]$ áp dụng **độc lập lên từng pixel**.

Định nghĩa toán tử điểm:

$$g[r, c] = f\big(I[r, c]\big) \quad \forall (r, c)$$

- f **không phụ thuộc** vào vị trí (r, c)
- f **không phụ thuộc** vào pixel lân cận
- Chi phí: $O(H \cdot W)$



Đồ thị của f (trục X = đầu vào, trục Y = đầu ra) được gọi là **hàm truyền** — hoàn toàn xác định hiệu ứng thị giác.

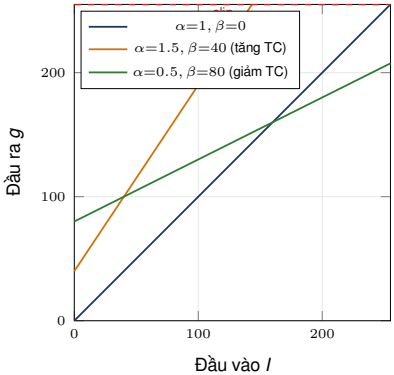
Toán Tử Tuyến Tính: $g = \alpha I + \beta$

Dạng tổng quát tuyến tính:

$$g = \alpha \cdot I + \beta, \qquad g = \texttt{clip}(g,\, 0,\, 255)$$

Ý nghĩa hình học của hàm truyền:

- $\alpha =$ **độ dốc** (slope) $\rightarrow$ **tương phản**
  $\alpha > 1$: dốc hơn $\rightarrow$ kéo giãn khoảng giá trị
  $\alpha < 1$: thoải hơn $\rightarrow$ thu hẹp khoảng giá trị
- $\beta =$ **dịch đứng** (intercept) $\rightarrow$ **độ sáng**
  $\beta > 0$: dịch lên $\rightarrow$ sáng hơn đều
  $\beta < 0$: dịch xuống $\rightarrow$ tối hơn đều



Hiệu Chỉnh Gamma — Tại Sao Phi Tuyến Tốt Hơn?

Câu hỏi dẫn dắt

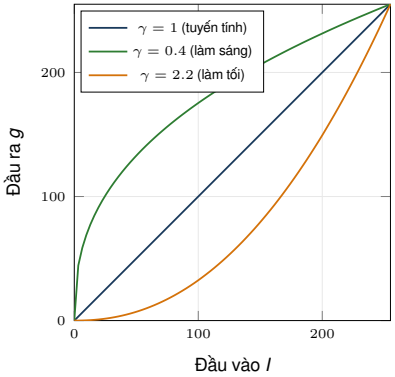
Tại sao bộ lọc tuyến tính không đủ để điều chỉnh ảnh tối?

Toán tử gamma (power-law):

$$g = \left(\frac{I}{255}\right)^{\gamma} \times 255$$

Đạo hàm (tốc độ khuếch đại tại mỗi mức sáng):

$$\frac{dg}{dI} = \gamma \cdot \left(\frac{I}{255}\right)^{\gamma-1}$$



Khi $\gamma < 1$: $\frac{dg}{dI}\big|_{I=0} = +\infty$ — **khuếch đại mạnh vùng tối**.

Khi $\gamma > 1$: ngược lại — **nén vùng tối**.

Cân Bằng Histogram — Hàm Truyền Tự Động

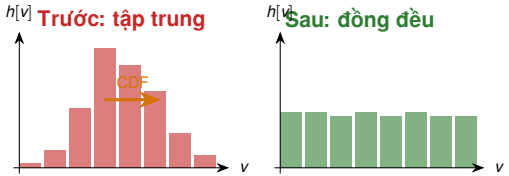
Câu hỏi dẫn dắt

Nếu không biết ảnh tối ở mức nào, làm sao chọn α và β?

- Ảnh kém = giá trị pixel **dồn cục** ở vùng hẹp
- Ảnh tốt = giá trị pixel **trải đều** trên $[0, 255]$
- Mục tiêu: tự động xây hàm truyền f để trải đều histogram

Công cụ toán học — CDF:

$$\text{CDF}[v] = \sum_{u=0}^{v} h[u]$$



CDF tăng đơn điệu $\Rightarrow$ dùng được làm hàm ánh xạ.

Công thức chuẩn hóa:

$$g = \left\lfloor \frac{\text{CDF}[I] - \text{CDF}_{\min}}{HW - \text{CDF}_{\min}} \times 255 \right\rfloor$$

Tại sao dùng CDF? Vì CDF tăng đơn điệu từ 0 đến HW — sau khi chuẩn hóa nó trở thành hàm ánh xạ từ $[0, 255]$ sang $[0, 255]$ trải đều

Demo 2

Toán Tử Điểm Ảnh

So sánh α, β khác nhau · Gamma $\gamma = 0.4$ và 2.2

Histogram trước và sau cân bằng CDF

Lọc Không Gian — Cần Nhìn Vào Láng Giềng

Giới Hạn Của Toán Tử Điểm

Câu hỏi dẫn dắt

Tôi muốn làm mờ ảnh. Dùng $g = \alpha I + \beta$ được không?

Phân tích: làm mờ = ánh sáng **lan sang pixel bên cạnh**.

Mà toán tử điểm $g[r, c] = f(I[r, c])$ chỉ nhìn vào chính pixel đó — **không thể** mang thông tin từ nơi khác đến.



Kết luận

Mọi hiệu ứng cần **ngữ cảnh không gian** (blur, sharpen, edge detect) đều **không thể** làm bằng toán tử điểm. Cần một lớp toán tử mới.

Đạo Hàm Rời Rạc — Từ Giải Tích Đến Ảnh Số

Liên hệ Giải Tích

Trong Giải Tích bạn đã học:

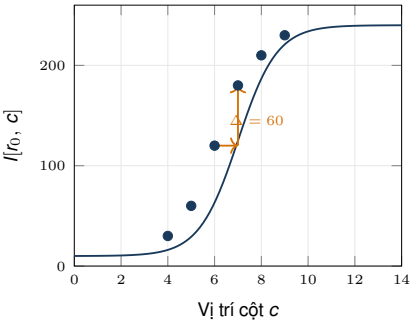
$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

Vấn đề: với ảnh số, $I[r, c]$ chỉ xác định tại các điểm nguyên. Bước nhỏ nhất là $h = 1$. **Không thể lấy giới hạn** $h \to 0$.

Xấp xỉ rời rạc — 3 phiên bản:

$$\text{Tiến:} \quad \left.\frac{\partial I}{\partial c}\right|_{r,c} \approx I[r, c+1] - I[r, c]$$

$$\text{Lùi:} \quad \left.\frac{\partial I}{\partial c}\right| \approx I[r, c] - I[r, c-1]$$



Đạo hàm lớn ⇔ cường độ thay đổi nhanh ⇔ **đây là vị trí cạnh!**

Đạo Hàm Bậc Hai Rời Rạc — Dẫn Xuất Từ Taylor

Liên hệ Giải Tích

Khai triển Taylor bậc 2 tại điểm x (bạn đã học ở Giải Tích 2):

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2}f''(x) + O(h^3)$$
$$f(x-h) = f(x) - hf'(x) + \frac{h^2}{2}f''(x) + O(h^3)$$

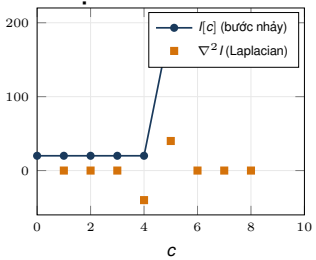
Cộng hai phương trình:

$$f(x+h) + f(x-h) = 2f(x) + h^2 f''(x) + O(h^4)$$

Giải ra $f''(x)$, đặt $h = 1$:

$$\boxed{f''(x) \approx f(x-1) - 2f(x) + f(x+1)}$$

Minh họa 1D: tín hiệu bước nhảy và đạo hàm bậc hai của nó:



*Laplacian $\neq 0$ chỉ tại **biên cạnh** — đây là ý nghĩa hình học của nó!*

Tích Chập — Cách Áp Dụng Bộ Lọc Lên Ảnh

Ý tưởng từ trung bình trượt:

$$g[i,j] = \frac{1}{9} \sum_{m=-1}^{1} \sum_{n=-1}^{1} I[i+m, j+n]$$

Tổng quát hóa: mỗi vị trí (m,n) có thể có **trọng số khác nhau** $K[m,n]$:

$$(I*K)[i,j] = \sum_{m=-k}^{k} \sum_{n=-k}^{k} I[i+m,\ j+n] \cdot K[m,$$

Đây là **tích chập rời rạc 2D** — phép toán trung tâm của xử lý ảnh.

Cơ chế

Trượt K qua mọi vị trí (i,j) trong ảnh. Tại mỗi vị trí: tính tổng trọng số của cửa sổ lân cận, ghi vào ảnh ra.

Ảnh đầu vào

10	10	10	10	10
10	10	10	10	10
10	10	**200**	10	10
10	10	10	10	10
10	10	10	10	10

Nhân K (3×3)

$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$
$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$
$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$

Ảnh đầu ra

10	10	10	10	10
10	27	27	27	10
10	27	27	27	10
10	27	27	27	10
10	10	10	10	10

$200/9 + 8 \times 10/9 \approx 27$

Tích Chập — Minh Họa “Trượt Và Tính”

Ảnh: trái tối ($= 10$), phải sáng ($= 40$)

10	10	10	40	40	40
10	10	10	40	40	40
10	10	10	40	40	40
10	10	10	40	40	40
10	10	10	40	40	40
10	10	10	40	40	40

Nhân hộp 3×3

$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$
$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$
$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$

Kết quả

	10	20	30		

Cửa sổ trượt qua ranh giới: $10 \to 20 \to 30$ — cạnh mờ dần!

Kết nối thị giác: pixel tại ranh giới sáng-tối nhận trung bình cộng của hai phía $\Rightarrow$ cạnh không còn đột ngột $\Rightarrow$ ảnh mờ.

Điều Kiện Biên

Đệm zero

Vùng ngoài $= 0$.

+ Đơn giản
– Viền tối ở mép

Phản chiếu

Gương pixel biên.

+ Không artifact
+ Chất lượng tốt nhất

Lặp lại

Kéo dài pixel biên.

+ Nền đồng đều

Ảnh gốc

A	**B**	**C**
D	**E**	**F**
G	**H**	**I**

Đệm zero

0	0	0	0	0
0	A	B	C	0
0	D	E	F	0
0	G	H	I	0
0	0	0	0	0

Phản chiếu

I	H	G	H	I
F	E	D	E	F
C	B	A	B	C
F	E	D	E	F
I	H	G	H	I

Tại Sao Gaussian? — Nhìn Từ Xác Suất

Liên hệ Giải Tích

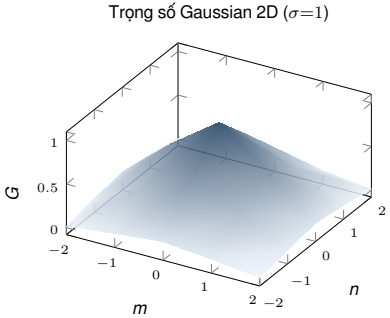
Phân phối chuẩn (normal distribution) trong Giải Tích/Xác Suất:

$$\mathcal{N}(\mu, \sigma^2): \quad p(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Nhân Gaussian 2D chính là PDF của phân phối chuẩn 2D tại $\mu = (0, 0)$:

$$G(m, n; \sigma) = \frac{1}{2\pi\sigma^2} e^{-\frac{m^2+n^2}{2\sigma^2}}$$

Ý nghĩa: trọng số giảm dần theo **khoảng cách** từ tâm phản ánh đúng cách mờ vật lý xảy ra trong thực tế (tán xạ ánh sáng)



Nhân 3×3 ($\sigma \approx 0.85$):

$$K_G = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Tính Tách Biệt Của Gaussian — Tiết Kiệm Chi Phí

Quan sát đại số:

$$G(m,n) = \frac{1}{2\pi\sigma^2} e^{-\frac{m^2+n^2}{2\sigma^2}} = \underbrace{\frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{m^2}{2\sigma^2}}}_{G_1(m)} \cdot \underbrace{\frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{n^2}{2\sigma^2}}}_{G_1(n)}$$

Vì $e^{-(m^2+n^2)} = e^{-m^2} \cdot e^{-n^2}$ — mũ tổng tách thành tích!

Hệ quả cho tích chập:

$$I * G_{2D} = (I * G_{\text{hàng}}) * G_{\text{cột}}$$

Minh họa bằng số:

$$K_G = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} = \frac{1}{4}\begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \cdot \frac{1}{4}\begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$$

So sánh chi phí:

Phương pháp	Phép nhân/pixel	$k = 15$
Tích chập 2D trực tiếp	k^2	225
Hai lần 1D (tách biệt)	$2k$	30
Tiết kiệm		**87%**

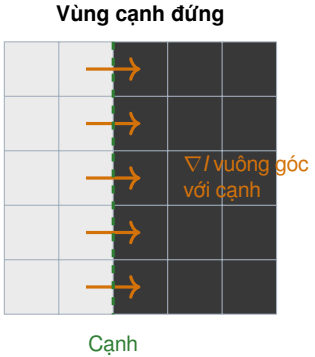
Gradient 2D — Hướng và Độ Mạnh Của Cạnh

Liên hệ Giải Tích

Trong Giải Tích bạn đã học **gradient** của hàm hai biến:

$$\nabla f(x, y) = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right)$$

Gradient chỉ **hướng tăng nhanh nhất** của hàm.



Gradient của ảnh:

$$\nabla I[r, c] = \left(\frac{\partial I}{\partial c}, \frac{\partial I}{\partial r} \right)$$

Gradient lớn $\Leftrightarrow$ cạnh mạnh.
Hướng gradient $\perp$ đường cạnh.

Bộ Lọc Sobel — Xấp Xỉ Gradient Bằng Tích Chập

Mục tiêu: tính $\partial I/\partial c$ và $\partial I/\partial r$ bằng tích chập.

Đạo hàm trung tâm theo c:

$$\frac{\partial I}{\partial c}[r, c] \approx \frac{I[r, c{+}1] - I[r, c{-}1]}{2}$$

Đây tương ứng nhân $[-1,\ 0,\ +1]$.

Sobel tăng cường bằng cách làm mờ nhẹ theo hướng **vuông góc** (giảm nhiễu) — tích ngoài (outer product) hai bộ lọc 1D:

$$S_x = \underbrace{\begin{bmatrix}1\\2\\1\end{bmatrix}}_{\text{làm mờ dọc}} \cdot \underbrace{\begin{bmatrix}-1 & 0 & +1\end{bmatrix}}_{\text{đạo hàm ngang}} = \begin{bmatrix}-1 & 0 & 1\\-2 & 0 & 2\\-1 & 0 & 1\end{bmatrix}$$

Hàng ảnh qua cạnh



S_x lớn tại vị trí cạnh

Tổng trọng số $S_x = 0 \Rightarrow$ output $= 0$ trên vùng đồng đều

$S_y = 0$ ở đây vì cạnh **đứng**
(S_y phát hiện cạnh **ngang**)

Làm Sắc Nét — Cộng Laplacian Vào Ảnh Gốc

Từ kết quả đã dẫn xuất:

$$\nabla^2 I[r,c] \approx I[r-1,c]+I[r+1,c]+I[r,c-1]+I[r,c+1]-4I[r,c]$$

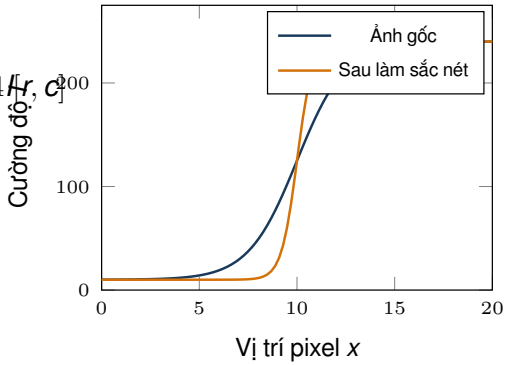
Ý tưởng làm sắc nét: Laplacian lớn tại cạnh
$\Rightarrow$ cộng nó vào ảnh để **tăng cường cạnh**:

$$g = I - \lambda \cdot \nabla^2 I \quad (\lambda > 0)$$

Với $\lambda = 1$, ghép thành một nhân duy nhất:

$$g = (I_{\text{kern}} - K_{\nabla^2}) * I = K_{\text{sắc}} * I$$

$$K_{\text{sắc}} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$



Nhân Laplacian:

$$K_{\nabla^2} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Bảng Tổng Hợp: Các Nhân Lọc và Ý Nghĩa Toán Học

Nhân	Tính toán	Tổng K	Hiệu ứng
Bộ lọc hộp	Trung bình đều $\frac{1}{k^2}\sum$	1	Làm mờ (artifact)
Gaussian	PDF chuẩn 2D rời rạc	1	Làm mờ (mượt)
Làm sắc nét	$I - \nabla^2 I$	1	Tăng cường cạnh
Sobel X	$\partial I/\partial c$ (trung tâm)	0	Cạnh đứng
Sobel Y	$\partial I/\partial r$ (trung tâm)	0	Cạnh ngang

Tổng trọng số = 1

Bộ lọc **bảo toàn độ sáng trung bình**. Vùng đồng đều (constant) cho output bằng chính nó: $\sum K \cdot c = c \cdot \sum K = c$.

Tổng trọng số = 0

Bộ lọc **phát hiện thay đổi**. Vùng đồng đều (constant) cho output = 0: $\sum K \cdot c = c \cdot \sum K = 0$. Chỉ nơi có gradient mới cho output $\neq 0$.

Demo 3

Lọc Không Gian: Thấy Toán Học Trong Ảnh

Box vs. Gaussian: profile cường độ tại cạnh

Sobel X và Sobel Y: phát hiện cạnh theo hướng

Làm sắc nét: so sánh profile trước/sau

Khử Nhiễu — Khi Nào Tuyến Tính Không Đủ?

Phân Tích Nhiễu — Kỳ Vọng và Phương Sai

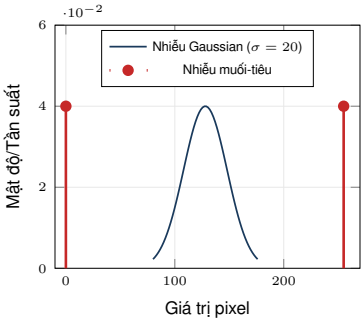
Liên hệ Giải Tích

Trong xác suất thống kê (học song song với GT2):

$$\text{Kỳ vọng:} \quad E[X] = \sum_x x \cdot p(x)$$

$$\text{Phương sai:} \quad \text{Var}[X] = E[(X - \mu)^2]$$

Phương sai đo mức độ **phân tán** của dữ liệu.



Nhiễu Gaussian: phân phối tập trung. Nhiễu muối-tiêu: hai đỉnh cực đoan.

Hai loại nhiễu chính:

- **Nhiễu muối-tiêu (salt-and-pepper):**
  Một số pixel nhảy đột ngột lên 0 hoặc 255.
  Phương sai **rất lớn** tại các điểm nhiễu.

Gaussian Lọc Được Nhiễu Nào?

Câu hỏi dẫn dắt

Ảnh bị nhiễu muối-tiêu. Dùng bộ lọc Gaussian có được không?

Phân tích bằng kỳ vọng:

$$g[r,c] = \sum_{m,n} K[m,n] \cdot I_{\text{noisy}}[r+m, c+n]$$

Nếu một pixel láng giềng là nhiễu với giá trị $v_{\text{noise}} = 255$:

$$g[r,c] \approx K[0,0] \cdot 255 + \sum_{(m,n)\neq(0,0)} K[m,n] \cdot v_{\text{norm}}$$

$K[0,0] = 4/16$ với Gaussian 3×3 $\Rightarrow$ nhiễu **vẫn đóng góp** $\frac{4}{16} \times 255 \approx 64$ vào output!

Hàng có nhiễu

82	80	255	81	79	83

↓ Gaussian

83	106	130	109	82	81

Nhiễu vẫn còn

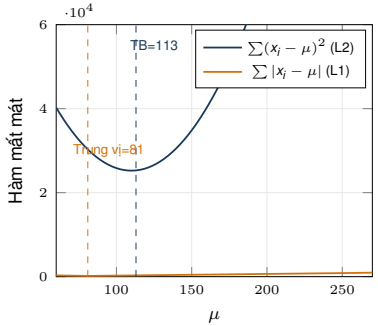
↓ Lọc trung vị

81	81	81	81	80	81

Nhiễu biến mất!

Trung Vị và L_1 — Tại Sao Mạnh Với Ngoại Lệ?

Tại sao L_1 mạnh hơn với nhiễu cực đoan?
Hàm mất mát L_2: $(x_i - \mu)^2$ — phạt **bình phương** sai lệch.
Một điểm ngoại lệ $x_i = 255$ khi $\mu \approx 80$:
L_2: phạt $(255 - 80)^2 = 30625$ — **rất lớn**, kéo μ lên mạnh.
L_1: phạt $|255 - 80| = 175$ — tuyến tính, ảnh hưởng vừa phải.



L2 bị kéo bởi 255. L1 bền vững hơn.

Kết luận

Trung vị (L_1) **bỏ qua** ngoại lệ tốt hơn trung bình (L_2). Đây là lý do toán học sâu nhất giải thích tại sao lọc trung vị mạnh với nhiễu muối-tiêu.

Bộ Lọc Trung Vị — Thuật Toán và Tính Phi Tuyến

Thuật toán:

1. Lấy k^2 giá trị trong cửa sổ $k \times k$
2. Sắp xếp tăng dần: $v_1 \leq v_2 \leq \cdots \leq v_{k^2}$
3. Kết quả $= v_{\lceil k^2/2 \rceil}$ (phần tử giữa)

Tại sao nhiễu không bao giờ là trung vị?

Với $k = 3$: cửa sổ 3×3 có 9 giá trị. Nhiễu muối-tiêu tạo giá trị 0 hoặc 255 — luôn ở vị trí số 1 hoặc số 9 trong dãy sắp xếp. Trung vị là phần tử **số 5**. Cần ít nhất **5 điểm nhiễu** trong cùng một cửa sổ để qua mặt trung vị.

Phi tuyến — không thể tích chập

Lọc trung vị **không thể** viết dưới dạng $(I * K)[i, j]$ vì nó dùng phép **sắp xếp**, không phải tổng tích vô hướng.

80	82	255	81	79
83	81	80	82	80
79	0	81	80	82
81	80	82	81	80
80	82	80	79	81

Cửa sổ tại $[1,$

Sắp xếp 9 giá

$0, 79, 80, \mathbf{81},$
$82, 82, 83, 25$

Trung vị $= v_5 =$

Trước: nhiễu (đỏ)

80	81	81	81	80
81	81	81	81	80
80	80	81	81	81
81	80	81	81	80
80	81	80	80	81

Sau: nhiễu biến mất

Demo 4

Nhiễu: Gaussian vs. Trung Vị

Nhiễu muối-tiêu: thay đổi mật độ và quan sát MSE

So sánh L_1 (trung vị) vs L_2 (Gaussian) bằng đồ thị

Thêm nhiễu Gaussian: lần này Gaussian thắng không?

Biến Đổi Hình Học — Ánh Xạ Vị Trí Pixel

Liên hệ Giải Tích

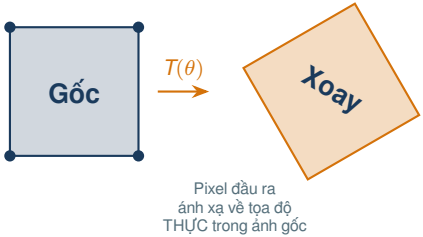
Đại số tuyến tính (học song song): một **ánh xạ tuyến tính** $T : \mathbb{R}^2 \to \mathbb{R}^2$ biểu diễn bằng ma trận 2×2:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Biến đổi hình học áp dụng ánh xạ này lên **mọi vị trí pixel** (r, c).

Phân biệt với toán tử điểm:

- Toán tử điểm: thay đổi **giá trị** $I[r, c]$, giữ vị trí
- Biến đổi hình học: thay đổi **vị trí** (r, c), giữ giá trị



Vấn đề với tịnh tiến:
$x' = x + t_x$ **không** phải ánh xạ tuyến tính (vì $T(0) \neq 0$). Cần mở rộng!

Tọa Độ Thuần Nhất — Mở Rộng Để Thống Nhất Mọi Biến Đổi

Ý tưởng: thêm một chiều thứ ba bằng 1:

$$(x, y) \mapsto (x, y, 1)$$

Giờ **mọi biến đổi** — kể cả tịnh tiến — đều là nhân ma trận 3×3:

$$(x', y', 1) = M \cdot (x, y, 1)$$

Tịnh tiến (không phải tuyến tính, nhưng biểu diễn được!):

$$\begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}}_{T} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Tịnh tiến (t_x, t_y): $\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$

2 DOF, bảo toàn: khoảng cách

Tỉ lệ (s_x, s_y): $\begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$

2–4 DOF, bảo toàn: góc

Xoay θ: $\begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$

1 DOF, bảo toàn: độ dài

Affine tổng quát: $\begin{bmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix}$

6 DOF, bảo toàn: đường thẳng song song

Dẫn Xuất Ma Trận Xoay Từ Tọa Độ Cực

Điểm $P = (x, y)$, bán kính r, góc ϕ so với trục Ox. Xoay thêm θ ngược chiều kim đồng hồ $\Rightarrow P' = (x', y')$.

Biểu diễn cực: $x = r\cos\phi$, $y = r\sin\phi$

Tính x' (dùng công thức cộng góc lượng giác từ GT1):

$$\begin{aligned} x' &= r\cos(\phi + \theta) \\ &= r\cos\phi\cos\theta - r\sin\phi\sin\theta \\ &= x\cos\theta - y\sin\theta \end{aligned}$$

Tính y':

$$\begin{aligned} y' &= r\sin(\phi + \theta) \\ &= r\sin\phi\cos\theta + r\cos\phi\sin\theta \\ &= x\sin\theta + y\cos\theta \end{aligned}$$

Dạng ma trận:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \underbrace{\begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}}_{R(\theta)} \begin{bmatrix} x \\ y \end{bmatrix}$$

Kiểm tra: $\det R = \cos^2\theta + \sin^2\theta = 1$ (bảo toàn diện tích), $R^{-1} = R^T$ (quay ngược).

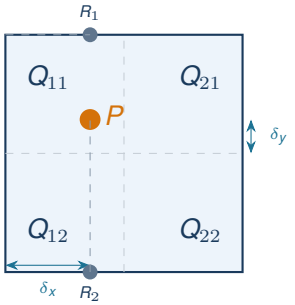
Warping Ngược và Nội Suy Song Tuyến

Vấn đề: sau khi xoay, pixel đầu ra (x', y') ánh xạ ngược về tọa độ **thực (không nguyên)** (x, y) trong ảnh gốc.

Tại sao warping ngược (inverse warping)?
Warping xuôi: pixel nguồn $\rightarrow$ pixel đích — có thể tạo **lỗ hổng** (nhiều pixel nguồn cùng ánh xạ vào một điểm, hoặc không pixel nào đến một điểm đích).
Warping ngược: pixel đích $\rightarrow$ tìm nguồn — **không có lỗ hổng**. **Nội suy song tuyến** — xấp xỉ I tại điểm thực (x, y):

$$R_1 = (1 - \delta_x)\, Q_{11} + \delta_x\, Q_{21}$$
$$R_2 = (1 - \delta_x)\, Q_{12} + \delta_x\, Q_{22}$$
$$P = (1 - \delta_y)\, R_1 + \delta_y\, R_2$$

Hai lần nội suy tuyến tính: theo x rồi theo y.



Liên hệ Giải Tích

Đây chính là **nội suy song tuyến** học trong Giải Tích số — xấp xỉ hàm bằng mặt phẳng trong ô đơn vị.

Demo 5

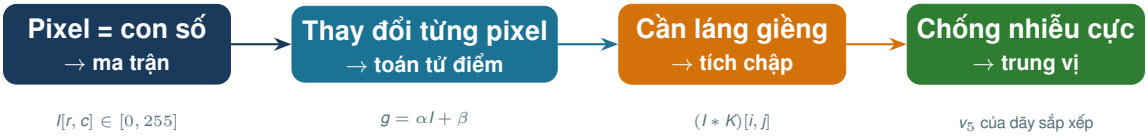
Xoay Ảnh Bằng Ma Trận Tự Xây

Xây dựng $R(\theta)$ thủ công trong NumPy

Warping ngược + nội suy song tuyến lên ảnh bàn cờ

So sánh nearest neighbor vs. bilinear: thấy aliasing

Nhìn Lại — Sợi Chỉ Xuyên Suốt Bài Học



Toán học đã học:

- Tích chập rời rạc 2D
- Phân tích Gaussian tách biệt
- Đạo hàm Laplacian → làm sắc nét
- Đạo hàm Sobel → phát hiện cạnh
- Ma trận xoay từ tọa độ cực
- Nội suy song tuyến tính

Kết nối thị giác:

- Nhân trượt qua cạnh → cạnh mờ dần
- Sobel lớn tại cạnh → phát hiện vị trí cạnh
- Trung vị loại ngoại lệ → sạch hơn Gaussian
- Warping ngược tránh lỗ hổng pixel

Câu Hỏi?

Xử Lý Ảnh và Thị Giác Máy Tính — 121036

Trường ĐH Giao Thông Vận Tải TP. Hồ Chí Minh